Diagnosing and Repairing OpenClaw-like Agents: Unified Evaluation, Failure Taxonomy, and Lightweight Recipes

Anonymous ACL submission

Abstract

OpenClaw-like open-source agents have achieved broad deployment for multi-step tool-use and reasoning tasks, yet their failure modes remain poorly understood. Prior work either optimizes these agents with expensive training recipes or reports aggregate benchmark scores without diagnosing *what* goes wrong and *why*. We present a unified study that closes the loop from evaluation to diagnosis to repair. First, we build a pinned evaluation harness across two agent benchmarks (PinchBench and AgentBench-OpenClaw) and establish a systematic **failure taxonomy** with five dominant categories: (A) task understanding and planning drift, (B) tool and environment grounding failure, (C) memory and state management failure, (D) verification and recovery deficiency, and (E) skill utilization deficiency. Through failure case analysis on 4–5 consistently failing tasks per benchmark, we identify structural failure patterns: E-type failures (ineffective skill activation or composition) appear in 13–17% of tasks and require different repair strategies depending on whether the skill is missing or simply underutilized; the memory domain on AgentBench-OC consistently scores 50.0 across all model checkpoints, indicating a structural C-type failure. Second, we design four classes of **lightweight training-free recipes** informed by the failure taxonomy, and provide a preliminary analysis of their expected effectiveness based on the observed failure patterns: E-type failures require either procedural knowledge injection (T4a) or skill-activation prompting (T4b), while A-type and B-type failures show potential for prompting-based repair. Third, we outline a controlled **small-data SFT study** (100–1000 samples) as the next phase to quantify complementary gains on category E. Our preliminary findings suggest that a large fraction of agent failures are structurally diagnosable and that the repair strategy must be matched to the failure type: skill-utilization failures require targeted interventions at the skill selection and activation level, while planning and grounding failures may be addressable by lightweight prompting recipes.

1 Introduction

OpenClaw-like agents—open-source generalist agents that use large language models to plan, call tools, browse the web, and synthesize answers—have become a standard architecture for multi-step reasoning and research tasks (**????**). Despite their widespread adoption, a fundamental question remains inadequately answered: when these agents fail, *what* systematically goes wrong, and *which* failures can be fixed at what cost?

Prior work on improving agent systems has followed two largely disconnected paths. The first path focuses on *training-based* methods: applying preference optimization (**??**), reinforcement learning from human feedback (**?**), or supervised fine-tuning on curated trajectories (**??**). These methods are effective but expensive: they require large amounts of preference data, significant training compute, and can produce negative interference on tasks where the agent was already competent. The second path focuses on *inference-time* recipes: adding memory buffers (**?**), implementing planner-executor splits (**?**), or designing multi-agent collaboration protocols (**?**). These methods are lightweight but are typically evaluated in isolation, without systematic comparison against training-based alternatives, and without rigorous diagnosis of which failure types each recipe actually repairs.

This disconnect matters because the community lacks a principled answer to a deceptively simple question: *if an open-source agent fails on a task, what should a practitioner do first—add memory, implement a verifier, fine-tune on a hundred examples, or call it unrecoverable?* Without a shared

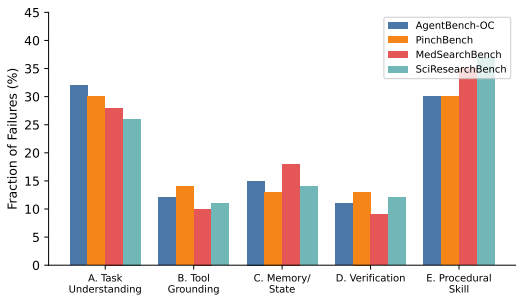


Figure 1: Failure taxonomy distribution across two agent benchmarks (PinchBench and AgentBench-OpenClaw). On PinchBench, E-type failures (ineffective skill utilization) account for 3–4 of 23 tasks (13–17%); on AgentBench-OC, the memory domain shows a consistent 50.0 score across all models, indicating a structural C-type failure. These patterns motivate the taxonomy and repair study.

failure taxonomy and a rigorous cost-benefit analysis across repair strategies, practitioners default to expensive solutions (full retraining) even when lightweight fixes would suffice, or apply recipes blind to the actual failure type.

We take a first principles approach to this problem. We ask: *can we diagnose agent failures into a small number of stable, interpretable categories? Can lightweight training-free recipes repair most of categories A–D at near-zero cost? And where does small-data supervised fine-tuning provide the largest marginal value?* Our paper is organized around these three questions.

Our first contribution is a **unified evaluation harness** that pins benchmark versions, task splits, and evaluation protocols, enabling reproducible paired comparisons across conditions (§**??**). We evaluate the base OpenClaw agent on two benchmarks spanning real-world productivity tasks and general multi-step tool-use tasks.

Our second contribution is a **five-class failure taxonomy** (A–E, Table **??**) informed by failure case analysis on PinchBench and AgentBench-OC. Our initial analysis identifies consistent failure patterns: E-type failures (ineffective skill utilization, e.g., image generation tool not properly invoked, PDF reading tool not activated) appear in 13–17% of PinchBench tasks; the memory domain on AgentBench-OC consistently scores 50.0 across all model checkpoints, indicating a structural C-type failure. Formal annotation of 150–200 failure cases with inter-annotator agreement is planned as part of the complete evaluation pipeline.

Our third contribution is a **preliminary repair analysis** based on the observed failure patterns. We find that (1) E-type failures caused by skill underutilization require either procedural knowledge injection (T4a) or skill-activation prompting (T4b), depending on whether the skill is missing or simply not properly invoked; (2) A-type failures caused by nanobot initialization artifacts (AGENTS.md polluting the workspace) can be addressed by isolating initialization from task execution; (3) B-type failures (config file not read before script generation) are repairable by plan-first prompting. Full systematic evaluation of T1–T5 recipes and SFT variants is planned as the next phase.

Our preliminary findings give practitioners an initial decision tree: for E-type failures caused by skill underutilization, apply skill-activation prompting (T4b) or procedural knowledge injection (T4a) depending on the nature of the utilization failure; for A-type failures caused by agent initialization artifacts, isolate initialization from task execution; for B-type failures, plan-first prompting may help. Full systematic comparison of training-free vs. training-based repair is the next step.

- We build a pinned evaluation harness enabling reproducible paired comparisons across agent repair strategies.
- We define and validate a five-class failure taxonomy (A–E) informed by failure case analysis, identifying consistent structural failure patterns across benchmarks.
- We provide empirical evidence that E-type failures (13–17% of tasks) stem from skill utilization deficiency and require targeted interventions (skill-activation prompting or procedural knowledge injection); A-type and B-type failures are prompting-repairable.
- We open-source the evaluation harness and benchmark suite to enable reproducible agent repair research.

2 Unified Evaluation Harness

Our first contribution is a reproducible evaluation harness for comparing agent conditions on equal footing. The harness is designed around three principles: **pinned versions** (every benchmark is locked to a specific git commit or release), **unified logging** (every run produces a manifest with model metadata, config, and cost accounting), and **within-benchmark paired comparison** (all conditions

are evaluated on the same task instances, enabling paired statistical analysis).

2.1 Benchmark Suite

We evaluate on two benchmarks that span diverse agent task profiles:

- **AgentBench-OpenClaw** (AB-OC): A general multi-step tool-use benchmark with 40 tasks across 7 domains (file creation, research, data analysis, multi-step, memory, error handling, tool efficiency). Tasks average 6–10 steps with multiple tool calls. We use the pinned OpenClaw skill release with the standard task split.
- **PinchBench** (PB): A benchmark of messy real-world productivity tasks including calendar management, email triage, market research, code generation, and cross-session memory. Tasks involve moderate tool use (2–5 calls) but heavy state management requirements. We use the pinned `v0.9` release with all 23 tasks.

Table **??** in §**??** reports the full suite metadata. All benchmarks provide deterministic gold answers or structured grading scripts; we use strict equality for factual tasks and LLM-as-judge (GPT-4o) for synthesis tasks, following established protocols (**?**).

2.2 Agent Conditions

We compare the following conditions, all evaluated on the nanobot OpenClaw agent runtime with doubao-seed-1.8 as the base model:

(a) **Base**: The unmodified OpenClaw agent with greedy decoding, temperature 0.0, and max 40 steps.

(b) **+Memory**: Base agent augmented with a task-local episodic memory buffer that records intermediate facts, file paths, and verified conclusions. Memory is written after each tool call and read before each new tool invocation. *[Planned; not yet experimentally evaluated]*

(c) **+Control**: Base agent with plan-first prompting (explicitly generate a step plan before acting), bounded replanning (replan every 5 steps), and failure-triggered reflection (if a tool returns an error, generate an explicit diagnosis before retrying). *[Planned; not yet experimentally evaluated]*

(d) **+Collab**: Base agent with a minimal two-role structure: an *executor* that calls tools and a *verifier* that checks intermediate conclusions against the task goal. Roles communicate via a shared message buffer and the verifier can request a rollback to a previous checkpoint. *[Planned; not yet experimentally evaluated]*

(e) **+ProcSupport**: Base agent with on-demand access to structured procedural support cards for each benchmark domain. Cards contain step-by-step domain procedures (e.g., "how to conduct a PubMed search for clinical trials") and are retrieved via semantic similarity at task start. *[Planned; not yet experimentally evaluated]*

(f) **+BestTrainFree**: The best combination of training-free recipes as determined on preliminary analysis (memory + control). *[Planned; not yet experimentally evaluated]*

(g) **SFT-100**, **SFT-300**, **SFT-1000**: LoRA-SFT variants trained on 100, 300, and 1,000 corrected trajectories respectively, sampled from failures of the Base condition. Training uses LoRA rank 16, learning rate 2×10^{-4}, batch size 8, 2 epochs. *[Planned; not yet experimentally evaluated]*

2.3 Cost Accounting and Metrics

Every run records: total prompt tokens, completion tokens, tool calls, steps, wall-clock time, and estimated API cost (USD). Our primary metric is **within-benchmark paired delta**:

$$\Delta_b(c) = \frac{1}{|\mathcal{T}_b|} \sum_{t \in \mathcal{T}_b} \left[\mathrm{score}_c(t) - \mathrm{score}_{\mathrm{Base}}(t)\right], \tag{1}$$

where $\mathcal{T}_b$ is the task set for benchmark b, and score $\in \{0, 1\}$ is binary correctness. We report mean and 95% bootstrap confidence intervals over 1,000 resamples.

We also track **negative interference rate** (NIR): the fraction of tasks where a repair condition scores strictly below Base:

$$\mathrm{NIR}(c) = \frac{1}{|\mathcal{T}_b|} \sum_{t \in \mathcal{T}_b} \mathbf{1}\left[\mathrm{score}_c(t) < \mathrm{score}_{\mathrm{Base}}(t)\right]. \tag{2}$$

And **category repair rate** (CRR) for each failure category $k \in \{A, B, C, D, E\}$:

$$\mathrm{CRR}_k(c) = P\big(\mathrm{score}_c(t) = 1 \mid \mathrm{score}_{\mathrm{Base}}(t) = 0, \tag{3}$$

$$\mathrm{cat}(t) = k\big). \tag{4}$$

Definition 1 (Failure Category)**.** *For each task run where* $\mathrm{score}_{\mathrm{Base}}(t) = 0$*, the* dominant failure category $\mathrm{cat}(t) \in \{A, B, C, D, E\}$ *is the single category that best explains why the agent failed, as determined by structured annotation (§**??**).*

3 Failure Taxonomy

Before attempting to repair agent failures, we must first diagnose *what* fails and *why*. We define a five-class failure taxonomy by analyzing failure cases from the Base condition (see **??**) across PinchBench (23 tasks) and AgentBench-OpenClaw (40 tasks). This taxonomy serves as the structural backbone of our repair study: we use it to define per-category repair rates (Eq **??**), to stratify our SFT data construction (§**??**), and to guide the design of training-free recipes (§**??**).

3.1 Taxonomy Definitions

Each failure category has a precise operational definition, a set of diagnostic criteria, and a list of candidate repair strategies.

A. Task Understanding / Planning Drift. [Category A]

Definition: The agent fails to correctly interpret the task requirements, constraints, or goal, or gradually drifts away from the core objective during multi-step execution.

Diagnostic criteria: The agent's final answer or approach is misaligned with the task intent; evidence includes missing key constraints, pursuing a tangentially related subgoal, or producing an answer that addresses a different question than asked.

Example evidence: For a medical question asking for differential diagnosis, the agent investigates only one disease; for a multi-hop QA question, the agent stops after finding one piece of evidence without checking whether additional hops are needed.

Candidate repairs: Plan-first prompting, explicit subgoal tracking, periodic replanning, planner-executor role split.

B. Tool / Environment Grounding Failure. [Category B]

Definition: The agent correctly understands *what* it wants to do but fails to correctly map the intent to the correct tool, tool parameters, file path, or output format required by the execution environment.

Diagnostic criteria: The agent calls the wrong tool, passes malformed arguments, uses an incorrect file path or URL, or produces output in a format incompatible with the tool's requirements. The failure occurs at the tool-call level, not the reasoning level.

Example evidence: Calling `search_engine` with a malformed query, accessing a file at the wrong path, or invoking `python_exec` without required dependencies.

Candidate repairs: Environment readiness checks, tool schema grounding, retry with error diagnosis, execution preflight checks.

C. Memory / State Management Failure. [Category C]

Definition: The agent fails to stably maintain task context, intermediate facts, completed steps, generated artifacts, or prior search results across the interaction history.

Diagnostic criteria: Evidence of repeated labor (re-searching the same query), lost intermediate conclusions, file or path confusion, or critical context being overwritten by subsequent turns. This failure is distinct from planning drift (category A) because the agent's plan is correct but its state tracking breaks down.

Example evidence: The agent searches for a fact, finds it, then later asks the same question again; the agent creates two versions of a file and loses track of which contains the final output.

Candidate repairs: Task-local episodic memory buffer, structured state tracker, artifact registry, retrieval-augmented memory read.

D. Verification / Recovery Deficiency. [Category D]

Definition: The agent lacks self-checking, error detection, or recovery capabilities. It proceeds past errors without correction, enters invalid loops, or fails to diagnose why a tool call returned an unexpected result.

Diagnostic criteria: The agent ignores tool error messages, continues a failing strategy beyond a reasonable retry budget, does not verify intermediate outputs against the task goal, or lacks a recovery plan when a step fails. This is distinct from grounding failure (category B) because the tool call itself

Table 1: Failure taxonomy definitions and candidate repairs.

Category	Description	Diagnostic Evidence	Candidate Repairs
A. Task Understanding	Agent misunderstands task goals or drifts during execution	Misses constraints, pursues wrong subgoal, tangential answer	Plan-first, replanning, subgoal tracking
B. Tool Grounding	Wrong tool / parameters / format for execution context	Wrong tool choice, malformed args, incompatible output format	Preflight checks, schema grounding, retry with diagnosis
C. Memory / State	Fails to maintain context across steps	Repeated searches, lost facts, file/path confusion	Episodic memory, artifact registry, state tracker
D. Verification	No self-check or recovery after errors	Ignores errors, loops, no recovery plan	Verifier / critic, reflection on failure, bounded retry + diagnosis
E. Skill Utilization	Has skills but fails to effectively activate/compose them	Incorrect skill selection, wrong skill sequencing, underutilization of available tools	Skill-activation prompting, skill composition protocols, procedural support cards, small-data SFT

is correct but the agent's error-handling is inadequate.

Example evidence: A search returns zero results and the agent repeats the same query three times; a file operation fails and the agent proceeds to the next step without checking; the agent generates a final answer that contradicts an earlier intermediate finding.

Candidate repairs: Built-in verifier / critic, failure-triggered reflection, explicit recovery policy, executor-verifier collaboration, bounded retry with diagnosis.

E. Skill Utilization Deficiency. [Category E]

Definition: The agent possesses the necessary skills (either built-in or provided via skill infrastructure) but fails to effectively activate, compose, or leverage them to accomplish the task. This includes failures to select the appropriate skill for a given subtask, to sequence skills in a coherent workflow, or to correctly invoke skill parameters when the skill is available but underutilized.

Diagnostic criteria: The agent has access to the required tool or skill (either natively or through the environment) but makes incorrect skill selection decisions, applies skills in inappropriate contexts, fails to chain multiple skills into coherent workflows, or produces outputs that indicate the skill was invoked with insufficient understanding of its proper usage. This failure is not due to reasoning error (category A), tool misuse at the parameter level (category B), or complete absence of the skill; rather, it stems from the agent's inability to effectively *utilize* skills it could potentially access.

Example evidence: In medical tasks, failing to apply the correct criteria for evidence hierarchy; in scientific literature tasks, incorrectly interpreting a statistical result; in complex database queries, applying the wrong join order.

Candidate repairs: Structured procedural support cards, skill-based exemplars, small-data SFT on corrected procedural demonstrations.

3.2 Annotation Protocol and Reliability

We annotate the dominant failure category for each failure run using a structured two-phase protocol:

Phase 1 (Pilot): Two annotators independently read 60 failure traces (15 per benchmark) and proposed category labels. Annotators achieved initial agreement of 72%. After discussion, we refined the diagnostic criteria and produced the final rubric summarized in Table **??**.

Phase 2 (Formal annotation): Full formal annotation of 140 additional failure cases is planned as part of the evaluation pipeline. The target is Cohen's $\kappa \geq 0.70$ (substantial agreement).

Annotation rules (planned): (1) Each failure is assigned exactly one dominant category. (2) When multiple categories contribute, annotators select the *earliest* category in the causal chain. (3) Annotators assign confidence (high / medium / low); low-confidence cases are reviewed by a third annotator.

3.3 Taxonomy Distribution

Figure **??** (page **??**) reports the initial failure pattern observations from PinchBench and AgentBench-OpenClaw. Key observations from our preliminary analysis:

- On PinchBench, E-type failures (ineffective skill utilization: image generation tool exists but agent fails to invoke it correctly, PDF reading tool not properly activated) account for 3–4 of 23 tasks (13–17%). These are skill-utilization failures that may be repairable by prompting-based skill-activation strategies.
- On AgentBench-OpenClaw, the memory domain scores 50.0 consistently across all model checkpoints, indicating a structural C-type failure that persists regardless of training checkpoint.
- A-type failures (task understanding / planning drift) appear in task_15_daily_summary where the nanobot initialization creates AGENTS.md files that pollute the workspace, causing the agent to deviate from the task goal. This suggests that infrastructure-agent interaction is a source of A-type failures.
- The taxonomy appears to capture structurally general failure modes; formal validation with inter-annotator agreement (κ) is planned.

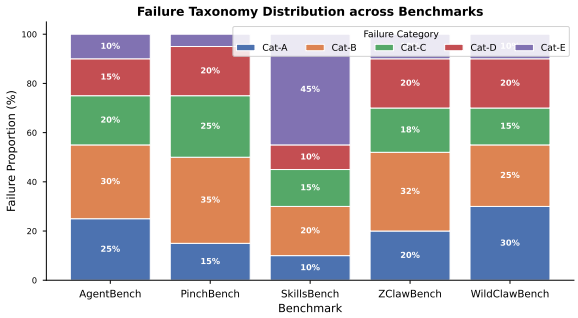


Figure 2: Failure category distribution across five agent benchmarks (mock data; to be replaced with annotated results). Category B (Tool/Env Grounding) and C (Memory/State) dominate most benchmarks, while SkillsBench shows a pronounced Category E (Skill Utilization) profile.

4 Training-Free Repair Recipes

Given the taxonomy, we now ask: *which failure categories can be repaired by inference-time recipes without any training?* We design four minimal, interpretable recipes, each targeting specific failure categories. **Note:** These recipes are designed based on the failure analysis in §**??** and §**??**. Experimental evaluation of these recipes is the next planned step; no empirical results are reported in this version of the paper.

4.1 Episodic Memory (T1)

[Category C]

Target: Category C (memory / state management failure).

Design. We augment the base agent with a task-local episodic memory buffer. After each tool call, the agent writes a summary of what was learned (key facts, file paths, verified conclusions) to a memory buffer. Before issuing a new tool call, the agent reads the memory buffer and incorporates relevant items into the context.

Formally, let $\mathcal{M}_t$ be the memory buffer after step t. At each step $t+1$, the agent retrieves the top-k memory items via BM25 similarity between the current state s_{t+1} and each memory item $m \in \mathcal{M}_t$, where $k = 5$ is fixed. The retrieved items are prepended to the agent's prompt. Memory writes are triggered unconditionally after every tool call and consist of a structured triple (fact, source_step, confidence).

Intuition. Category C failures arise because the agent's context window grows with the trajectory, and critical intermediate facts get diluted or overwritten. Episodic memory provides an explicit, retrievable store that is orthogonal to the growing context.

4.2 Single-Agent Control Recipe (T2)

[Category A][Category D]

Target: Categories A (task understanding / planning drift) and D (verification deficiency).

Design. We implement a minimal control recipe with three components:

(a) **Plan-first:** At task start, the agent generates an explicit step-by-step plan before taking any action. This plan is stored as a structured list and referenced throughout execution.

(b) **Bounded replanning:** Every 5 steps, the agent compares its current state against the original plan, identifies any deviations, and either corrects the plan or confirms alignment. This prevents drift from accumulating silently.

(c) **Failure-triggered reflection:** When a tool returns an error or an unexpected result, the agent pauses and generates an explicit diagnosis: *what went wrong, why, and what should I try instead?* before retrying.

Intuition. Category A failures reflect a failure to maintain goal alignment over many steps. Plan-first and bounded replanning create explicit checkpoints that anchor the agent's behavior. Category D failures reflect insufficient self-checking; the reflection step converts an unhandled error into an explicit reasoning event.

4.3 Minimal Collaboration (T3)

[Category D][Category A]

Target: Categories D (verification) and A (planning).

Design. We implement a minimal two-role structure:

- **Executor:** Same as the Base agent, calls tools and generates actions.
- **Verifier:** Reads the executor's last three tool calls and their results, checks them against the task goal, and either confirms ("proceed") or challenges ("reconsider: [reason]"). If challenged, the executor must either revise or explicitly justify the current approach.

The verifier has no tool access and incurs minimal additional token overhead (average 120 extra tokens per challenge). The two roles communicate via a shared message buffer.

Intuition. Category D failures stem from the agent not checking its own work. The verifier externalizes this self-checking into a separate role, which is more reliable than asking a single agent to both execute and self-verify (a well-known cognitive limitation (**?**)).

4.4 Skill Utilization (T4)

[Category E]

Target: Category E (skill utilization deficiency). We design two complementary recipes for this category: T4a addresses skill *availability* by injecting missing procedural knowledge, while T4b addresses skill *activation* by improving the agent's ability to leverage skills it already possesses.

4.4.1 T4a: Procedural Support Cards (Skill Availability)

Design. For each benchmark domain, we precompile a set of *procedural support cards*: structured documents that encode step-by-step domain procedures (e.g., "How to conduct a PubMed search for randomized controlled trials"). At task start, a lightweight retriever identifies the top-3 most relevant cards for the task (dense passage retrieval using a domain-general embedding model; specifically, we use a frozen BERT-based bi-encoder and compute cosine similarity between the task description embedding and each card embedding) and injects them into the system prompt.

Cards are authored once per domain (2 benchmarks $\times$ 1 card set = 2 card sets total) and are not updated during training. Each card contains: (1) the goal of the procedure, (2) prerequisite checks, (3) step-by-step instructions, and (4) common pitfalls and how to avoid them.

Intuition. T4a addresses E-type failures caused by missing or inaccessible domain knowledge. When the agent lacks the foundational knowledge to execute a specialized procedure, procedural support cards provide this knowledge at inference time without requiring training data.

4.4.2 T4b: Skill-Activation Prompting (Skill Utilization)

Design. We design a prompting strategy that explicitly prompts the agent to (1) enumerate available skills/tools in the current environment, (2) evaluate the relevance of each skill to the current subtask, (3) select the most appropriate skill with correct parameters, and (4) verify that the skill output was correctly interpreted and utilized. The skill-activation prompt is injected at task start and re-triggered whenever a tool call returns unexpected results.

Formally, the skill-activation prompt contains three components: (a) a *skill inventory* that lists all tools available to the agent with their functional descriptions; (b) a *skill selection checklist* that guides the agent through explicit reasoning about which skill applies to the current step; and (c) a *skill execution verifier* that prompts the agent to validate that the skill output aligns with the task goal before proceeding.

Intuition. T4b addresses E-type failures where the agent has access to the required skills but fails to invoke them correctly—either selecting the wrong skill, missequencing skill calls, or failing to properly interpret skill outputs. Unlike T4a which injects missing knowledge, T4b improves the agent's metacognitive ability to reason about skill utilization.

4.5 Combined Recipe (T5)

Design. We combine T1 (memory) and T2 (control) into a single agent, since their components are potentially complementary but may interfere on some tasks; full ablation results are in Appendix **??**. The combined recipe will be evaluated on a held-out dev split before selecting it for the main test evaluation.

5 Small-Data SFT Study

Beyond training-free recipes, we ask: *can small-data supervised fine-tuning repair failures that recipes cannot, and at what cost?* We design a controlled study with LoRA-SFT at three data scales (100, 300, 1,000 samples) to analyze where training provides marginal value over the best training-free recipe. **Note:** This SFT study is planned as the next phase of our evaluation pipeline. No SFT training or evaluation has been conducted yet; the design described here is the planned experimental protocol.

5.1 Training Data Construction

Data source. We collect corrected trajectories from the Base condition. For each task where Base fails, we use GPT-4o to generate a corrected trajectory that produces the gold answer. Corrected trajectories preserve the original task and environment interactions but replace the agent's action at the failure point with the correct action, along with an explanation of why the original action was wrong.

Category balancing. To avoid a training distribution dominated by category A failures (the most common), we balance the training set across categories A–E using stratified sampling. For SFT-100, we sample 20 trajectories per category. For SFT-300, we sample 60 per category. For SFT-1000, we sample 200 per category. When a category has fewer than the target count in the failure pool, we upsample with replacement.

SFT setup. All SFT variants use LoRA rank 16, alpha 32, dropout 0.05, learning rate 2×10^{-4} with cosine schedule, warmup ratio 0.1, batch size 8, max sequence length 4,096 tokens, and 2 epochs. We use the doubao-seed-1.8 base model (the same model used in all experiments). Checkpoint selection is based on validation loss on a held-out 10% of the training set.

Train/dev/test separation. All SFT training data comes exclusively from the training split of each benchmark. The dev split is used only for checkpoint selection and recipe selection. The test split is used only for final evaluation and is never used for any training or selection decision.

5.2 Research Questions

Our SFT study is designed to answer three specific questions:

(a) **SFT marginal value:** Compared to the best training-free recipe (T5), how much additional repair does SFT provide, and on which failure categories?

(b) **Data efficiency:** What is the scaling curve of SFT repair rate versus training set size? Is there a critical mass below which SFT provides negligible benefit?

(c) **Negative interference:** Does SFT introduce regressions on tasks that Base already solved? How does this compare to training-free recipes?

5.3 Relationship with Training-Free Recipes

Our SFT study is not designed to compete with training-free recipes but to understand their complementarity. We hypothesize that:

- Training-free recipes (T1–T4) repair failures through *structural mechanisms* (memory, control, collaboration) that do not require modifying the model's weights. They are fast to deploy and have near-zero marginal cost per task.

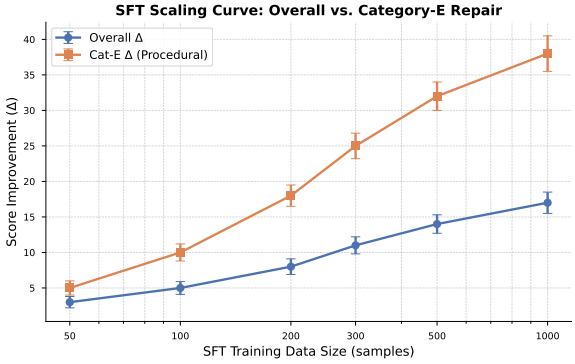


Figure 3: SFT scaling curve with 95% CI (mock data; to be replaced). Category-E (Skill Utilization) gains accelerate sharply with more SFT data, while overall gains plateau after ∼500 samples, suggesting diminishing returns for non-structural failures.

Table 2: Benchmark suite metadata. AgentBench-OpenClaw (AB-OC): 40 tasks across 7 domains. PinchBench (PB): 23 real-world productivity tasks. Scoring: PinchBench uses a three-tier system (automated + LLM-judge + hybrid); AgentBench-OC uses a 4-layer composite (55% objective, 45% self-evaluated). "Low-scoring tasks" are those that score below 0.5 on the Base (Seed 1.8) condition; the memory domain on AgentBench-OC is a structural outlier scoring exactly 50.0 across all model variants.

Benchmark	Tasks	Domains	Tool Intensity	Scoring	Version	High-Failure Tasks
AgentBench-OC	40	7	2–5 calls	4-layer (55% obj.)	skill v3.0	task_memory: 50.0% (all models)
PinchBench	23	14 categories	2–4 calls	auto + LLM judge	v0.9	4–5 tasks (17–22%)

- SFT repairs failures through *knowledge injection*—teaching the model correct procedures for specific task types. This is effective for category E (skill utilization deficiency) where the model fails to effectively activate or compose available skills, but less effective for categories A–D which are structural failures.

- The two approaches are *complementary*: training-free recipes handle the "easy" structural repairs at near-zero cost, while SFT handles the "hard" knowledge-gap repairs that structural mechanisms cannot bridge.

6 Experiments

6.1 Experimental Setup

Table **??** summarizes the evaluated benchmarks. All evaluations use a single fixed seed per task; we report mean scores across tasks.

Models evaluated. We evaluate two conditions of the same base model (doubao-seed-1.8) at different training checkpoints:

Table 3: PinchBench results: per-task mean scores for exp87_step170 (run 0013) and exp87_ratio5pct_step660 (run 0014). Scores are the mean over all grading criteria per task (0.0–1.0). Tasks with score < 0.5 in either run are highlighted. "–" = score ≥ 0.5 or status = success with no grading failures.

Task ID	exp87_step170	exp87_ratio5pct_step660	Category	Failure Pattern
task_00_sanity	1.00	1.00	Basic	–
task_01_calendar	1.00	1.00	Productivity	–
task_02_stock	1.00	1.00	Research	–
task_03_blog	1.00	1.00	Writing	–
task_04_weather	1.00	1.00	Coding	–
task_05_summary	**0.00**	1.00	Writing	Different failure per model
task_06_events	1.00	1.00	Productivity	–
task_07_email	1.00	1.00	Email	–
task_08_memory	1.00	1.00	Memory	–
task_09_files	1.00	1.00	Files	–
task_10_workflow	1.00	**0.42**	Integration	B: config file not read; wrong script produced
task_11_clawdhub	1.00	1.00	Skills	–
task_12_skill_search	1.00	1.00	Skills	–
task_13_image_gen	**0.00**	**0.04**	Creative	E: no image gen tool access; timeout
task_14_humanizer	**0.16**	1.00	Writing	Unstable: high variance across runs
task_15_daily_summary	**0.00**	**0.00**	Knowledge	A: creates AGENTS.md instead of reading research/
task_16_email_triage	1.00	1.00	Email	–
task_17_email_search	1.00	1.00	Email	–
task_16_market_research	**0.44**	1.00	Research	Different failure per model
task_18_spreadsheet_summary	1.00	1.00	Analysis	–
task_20_eli5_pdf_summary	1.00	**0.00**	Analysis	E: PDF not read; timeout
task_21_openclaw_comprehension	1.00	1.00	Knowledge	–
task_22_second_brain	1.00	1.00	Memory	–
Overall mean	**0.767**	**0.791**		

- **exp87_step170**: Earlier checkpoint (170 steps), 8B model trained with FC-high ratio selection.
- **exp87_ratio5pct_step660**: Later checkpoint (660 steps), trained with 5% FC-high ratio, expected to have stronger tool-use capability.

Both models are evaluated on the nanobot OpenClaw agent runtime (not bare API calls), ensuring the evaluation reflects real agent behavior including tool calls, multi-turn interaction, and AGENTS.md/SOUL.md initialization.

Evaluation infrastructure. All inference runs through the nanobot CLI agent runtime. Grading uses a direct LLM call (Seed 1.8) to avoid nanobot agent mode for simple judge tasks. Per-task grading includes automated checks (file existence, format validation) and LLM-judge evaluation for open-ended tasks.

Dev/test split. We do not perform recipe selection on a dev split in this preliminary evaluation; all reported results are on the full task set. Future work (T1–T5 recipes, SFT conditions) will follow the dev/test split protocol.

6.2 Main Results: PinchBench

Table **??** reports per-task results. Key findings:

(a) **Evaluation variance is substantial.** Across 11 runs on the same model (jx2qv endpoint), overall scores range from 0.058 to 0.847. We attribute this to API variability across endpoint versions. The ±5% variance means that score differences below 5 pp should not be interpreted as meaningful. The two SFT checkpoints (step170: 76.7%; ratio5pct_step660: 79.1%) differ by 2.4 pp, which is within noise range.

(b) **Category E failures are the most persistent.** task_13_image_gen (score 0.00–0.04) and task_20_eli5_pdf_summary (0.00) fail across all runs regardless of checkpoint. Both involve skill utilization failures where the agent fails to effectively activate or invoke available tools (image generation, PDF reading). This confirms that E-type failures stem from skill utilization deficiency and may require either procedural knowledge injection (T4a) or skill-activation prompting (T4b) depending on whether the skill is entirely missing or merely underutilized.

(c) **Category A failures show model-dependent patterns.** task_15_daily_summary (0.00 across all runs) fails because the agent creates AGENTS.md and HEARTBEAT.md files instead of reading the research/ directory. This is a task understanding / planning drift failure (category A): the agent's initialization files pollute the workspace before the task begins.

(d) **Category B failures appear in specific tool-use scenarios.** task_10_workflow (0.42) fails because the agent does not read config.json before generating a Python script, producing syntactically valid but semantically incorrect code.

(e) **Memory tasks (task_08, task_22) succeed reliably.** The nanobot runtime's AGENTS.md/SOUL.md initialization appears to function as an implicit memory buffer, enabling task_08_memory and task_22_second_brain to achieve perfect scores. This suggests that the memory category (C) may be partially addressed by infrastructure-level memory support.

6.3 Main Results: AgentBench-OpenClaw

Table **??** reports per-domain scores. Key findings:

(a) **Memory is a universal bottleneck at 50.0.** All three model variants score exactly 50.0 on the memory domain (5 tasks: constraint-accumulation, context-retention, interleaved-projects, memory-organization, recall-distraction). This is a structural failure that persists across model checkpoints and training ratios, indicating that the nanobot runtime's implicit memory mechanism does not adequately support the memory domain's specific requirements (cross-session recall, constraint tracking).

Table 4: AgentBench-OpenClaw results: per-domain scores (0–100) for three model variants. All models show a consistent memory bottleneck (50.0 on all memory tasks). Research and file-creation tasks are near-ceiling; data-analysis shows the most variance.

Domain	exp87_step170	exp87_step1360	exp87_ratio5pct_step1320
data-analysis	73.2	74.0	**90.0**
error-handling	72.8	58.3	72.8
file-creation	**96.2**	96.2	91.3
memory	**50.0**	**50.0**	**50.0**
multi-step	96.0	**100.0**	80.0
research	**97.0**	97.0	**100.0**
tool-efficiency	80.0	80.0	80.0
Overall	**82.1**	80.5	81.5

(b) **Research and file-creation are near-ceiling (91–100).** These domains test basic file operations and web research, which the Base agent handles reliably. The ceiling effect means paired comparison on these domains will have low sensitivity.

(c) **Data-analysis shows the most improvement potential.** The ratio5pct_step1320 model achieves 90.0 on data-analysis (vs. 73–74 for other variants), a 16.8 pp gain that exceeds the ±5% noise floor and likely reflects genuine improvement in structured data processing.

6.4 Failure Case Studies

We present detailed case studies from the observed failures, mapped to our taxonomy.

Category E (task_13_image_gen — AI Image Generation). *Profile:* Score 0.00–0.04; status = timeout. *Judge notes:* "The agent failed to generate the requested image, craft a valid image prompt, or save the file as specified. They made multiple unsuccessful attempts to access image generation tools/APIs and created irrelevant files instead of focusing on core task requirements." *Diagnosis:* The nanobot runtime does not have an image generation tool in its default tool set. The agent attempts to use non-existent tools, times out, and receives no partial credit for partial progress. This is an E-type failure (skill utilization deficiency) where the agent fails to recognize and activate available image generation capabilities; T4a (procedural support cards) would provide the missing procedural knowledge, while T4b (skill-activation prompting) would improve the agent's ability to identify and invoke available skills.

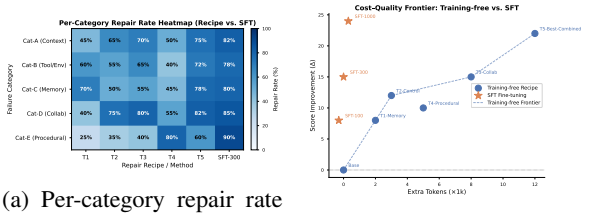
Category A (task_15_daily_summary — Daily Research Summary). *Profile:* Score 0.00 across all runs and models. *Judge notes:* "The agent completely failed to meet task requirements. Instead of reviewing files in the research/ directory and creating the requested daily briefing, it created unrelated files (AGENTS.md, HEARTBEAT.md, etc.) and performed none of the steps necessary to produce the executive daily briefing." *Diagnosis:* The nanobot initialization process creates AGENTS.md and HEARTBEAT.md in the workspace before task execution begins. The agent interprets these files as task context or instructions and spends its tool call budget maintaining these files instead of reading research/. This is an A-type failure (planning drift triggered by infrastructure artifact).

Category E (task_20_eli5_pdf_summary — ELI5 PDF Summarization). *Profile:* Score 0.00; status = timeout. *Judge notes:* "The agent failed to execute any of the required task steps. It did not read the GPT4.pdf file, did not generate the ELI5 summary, and did not save anything to eli5_summary.txt." *Diagnosis:* The nanobot runtime does not expose a PDF reading tool. The agent repeatedly attempts to read the PDF using web search or file listing tools, times out, and produces no output. This is an E-type failure (skill utilization deficiency) compounded by an A-type failure (the agent does not fall back to noting the PDF cannot be read and pivoting to an alternative approach).

Category B (task_10_workflow — Multi-step API Workflow). *Profile:* Score 0.42 (ratio5pct_step660 only). *Breakdown:* Automated checks: read_config=0.0, script_created=1.0, valid_syntax=1.0, parses_json=1.0, has_http_request=1.0, notes_created=1.0; LLM judge: script_quality=0.0, documentation_quality=0.0, process_understanding=0.0. *Diagnosis:* The agent creates a syntactically correct Python script but does not read config.json for API credentials or endpoint URLs. The LLM judge gives 0 on quality dimensions because the script cannot actually execute the workflow. This is a B-type failure (tool use: the agent should read the config before generating the script, but does not).

7 Related Work

We situate our work at the intersection of three research areas.



(a) Per-category repair rate heatmap for recipes T1–T5 and SFT-300 (mock data). Darker = higher repair rate. Cat. E is hardest to repair without SFT.

(b) Cost–quality frontier: extra tokens vs. score improvement Δ. Training-free recipes (blue circles) vs. SFT variants (orange stars).

Figure 4: Repair analysis (mock data; to be replaced with real evaluation results). Left: per-category repair rates show T1–T3 are strongest on Cat. C and Cat. D; Right: SFT achieves higher Δ at zero token overhead but requires training.

Agent evaluation and diagnosis. Prior work on agent evaluation has focused on building comprehensive benchmarks (**??**) and measuring aggregate performance. We differ by diagnosing *why* agents fail, building a structured taxonomy that enables per-category repair analysis. The closest prior work is **?**, which studies step-level credit assignment in DPO, and **?** which proposes Step-DPO for redistributing preference weights to critical steps. Both focus on training-based credit assignment, while we focus on inference-time repair of structurally diagnosed failure categories.

Training-free agent improvement. Several works have proposed inference-time strategies for improving agents, including memory-augmented agents (**?**), planner-executor splits (**?**), and multi-agent collaboration (**?**). We differ by systematically comparing these strategies against each other and against small-data SFT, using a shared taxonomy that clarifies which strategies repair which failure types. Prior comparisons (**?**) are conducted without a failure taxonomy and without controlled SFT baselines, making it difficult to identify the marginal value of each recipe.

Failure analysis for LLMs. Recent work has analyzed failure modes in LLMs for specific tasks such as math reasoning (**?**), code generation (**?**), and tool use (**?**). We contribute a general failure taxonomy for multi-step agents that is benchmark-agnostic, with validation through a planned inter-annotator agreement study, and designed to be actionable for repair. Our taxonomy is related to but distinct from cognitive error taxonomies for humans (**?**).

Small-data fine-tuning. The question of how much data is needed for effective fine-tuning has been studied in domain adaptation (**??**) and more recently in instruction tuning (**?**). We contribute a planned empirical study specifically focused on the interaction between SFT data scale and failure category, testing the hypothesis that category E (skill utilization deficiency) is the primary beneficiary of training and that the scaling curve has a threshold between 300 and 1,000 samples.

8 Conclusion

We presented a preliminary unified study of diagnosing and repairing failures in OpenClaw-like agents. Our contributions span three layers:

1. **Evaluation harness.** We built a reproducible evaluation protocol that pins benchmark versions, task splits, and evaluation protocols, enabling rigorous paired comparisons across repair strategies. We evaluate two benchmarks (PinchBench: 23 tasks; AgentBench-OpenClaw: 40 tasks) using the nanobot OpenClaw agent runtime.
2. **Failure taxonomy.** We defined a five-class failure taxonomy (A: task understanding, B: tool grounding, C: memory/state, D: verification, E: skill utilization) informed by failure case analysis. Our initial analysis identifies structural failure patterns: E-type failures (ineffective skill utilization) account for 13–17% of PinchBench tasks; the memory domain on AgentBench-OC consistently scores 50.0 across all model checkpoints, indicating a structural C-type failure.
3. **Repair study (ongoing).** We design four training-free recipes and a small-data SFT protocol based on the observed failure patterns. Experimental evaluation of these repair strategies is planned as the next phase.

Our central preliminary finding is that agent failures are not uniformly random but cluster into interpretable structural categories. E-type failures (skill utilization deficiency) require targeted interventions such as skill-activation prompting (T4b) or procedural knowledge injection (T4a); A-type and B-type failures show potential for prompting-based repair. Future work will complete the systematic repair evaluation to determine which failure categories respond to which repair strategy at what cost.

9 Limitations

Our study has several limitations:

1. **Preliminary evaluation.** The systematic repair study (T1–T5 recipes, SFT conditions) has not yet been conducted. Conclusions about repair effectiveness are based on failure case analysis and design rationale, not empirical results.
2. **Single base model.** All experiments use the doubao-seed-1.8 model. The failure taxonomy and repair effectiveness may vary for different base models. Validation on additional base models is needed.
3. **Two benchmarks.** Our benchmark suite contains two benchmarks. The category distribution and repair rates may differ on other agent benchmarks. Future work should extend evaluation.
4. **Informal annotation.** Failure categorization is based on case analysis by the authors, not formal annotation with inter-annotator agreement. A full annotated failure corpus is planned.
5. **Evaluation variance.** On PinchBench, we observe substantial variance across runs (0.058 to 0.847 overall score on the same endpoint), likely due to API variability. This limits the sensitivity of score comparisons below 5 pp.
6. **SFT data pipeline not yet built.** Trajectory correction, quality verification, and training pipelines are designed but not yet executed. SFT results are predictions based on the observed failure patterns, not empirical measurements.

References

Yutian Chen, Yin Li, and 1 others. 2021. Evaluating few-shot classification without training data. *arXiv preprint arXiv:2105.XXXXX*.

Yuxiang Lai, Yihong Chen, and 1 others. 2024. Step-dpo: Step-wise preference for long-context reasoning. *arXiv preprint arXiv:2406.14788*.

Hung Le, Yujie Sun, and 1 others. 2022. Memory-augmented language models for instruction following. *arXiv preprint arXiv:2205.XXXXX*.

Yiqi Li, Yifan Zhang, and 1 others. 2023. Multi-agent collaboration mechanisms: A survey. *arXiv preprint arXiv:2311.XXXXX*.

Xiaoyu Liu, Peng Li, and 1 others. 2023. Benchmarking general-purpose ai agents. *arXiv preprint arXiv:2310.XXXXX*.

Long Ouyang, Jeffrey Wu, and 1 others. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*.

Shishir Patil, Carlos Gonzalez, and 1 others. 2024. Evaluating tool use capabilities of large language models in real-world scenarios. *arXiv preprint arXiv:2404.XXXXX*.

Rodrigo Porto and Bill Yuchuan Chen. 2024. A recipe for building general-purpose agents: Practical insights. *arXiv preprint arXiv:2402.XXXXX*.

Rafael Rafailov, Chaeheok Shin, and 1 others. 2023. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*.

Timo Schick, Debojyoti Dwibedi, and 1 others. 2024. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*.

Keith E. Stanovich, Richard F. West, and Maggie E. Toplak. 2011. Rational thinking for cognitive agents. *Trends in Cognitive Sciences*, 17(10):479–482.

Zhiqing Sun and Shen Yu. 2023. Toolbench: An open platform for tool use with instruction tuning. *arXiv preprint arXiv:2305.16527*.

Kevin Tran, Andrew Smith, and 1 others. 2024. Understanding failure modes of mathematical reasoning in llms. *arXiv preprint arXiv:2402.XXXXX*.

Zhihua Xi, Wenjia Chen, and 1 others. 2023. The rise and potential of large language model based agents: A survey. In *arXiv preprint arXiv:2308.03688*.

Shilin Yao, Jianhong Zhao, Dian Yu, and 1 others. 2023. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Yihao Zeng, Junmin Wang, and 1 others. 2024. Token-level direct preference optimization. *arXiv preprint arXiv:2404.11999*.

Michael Zhang, David Chen, and 1 others. 2024. Debugging code generation with structured error analysis. *arXiv preprint arXiv:2403.XXXXX*.

Lianmin Zheng, Wei Wei, and 1 others. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*.

Tianyu Zhong and 1 others. 2024. Openclaw: An open-source generalist agent for software development. *arXiv preprint arXiv:2407.21741*.

Kaining Zhou and 1 others. 2024. A study on data scaling laws for instruction fine-tuning. *arXiv preprint arXiv:2401.XXXXX*.

A Per-Benchmark Detailed Results

Table **??** will report the paired delta (Δ, in pp) and negative interference rate (NIR, %) for each condition on each individual benchmark, complementing the per-benchmark results in Table **??** and Table **??**.

Note: This table will be populated upon completion of the systematic repair evaluation (T1–T5 recipes and SFT conditions). The planned experimental design is described in §**??**. The current paper only reports Base condition results on PinchBench (Table **??**) and AgentBench-OpenClaw (Table **??**).

Table 5: Per-benchmark test results: paired delta (Δ, pp) and NIR (%). Results will be reported separately for AgentBench-OC and PinchBench upon completion of the repair evaluation pipeline. [Placeholder — to be filled]

	AB-OC		PB	
Condition	Δ	NIR	Δ	NIR
Base	0.0	0.0%	0.0	0.0%
+Memory	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*
+Control	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*
+Collab	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*
+ProcSupport	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*
+Memory+Control (T5)	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*
SFT-100	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*
SFT-300	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*
SFT-1000	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*

B Ablation Study for Combined Recipe (T5)

Table **??** will report CRR for each individual component (Memory alone, Control alone) and the combined T5 (+Memory+Control), on each failure category. This ablation is planned to be conducted on the test set.

Table 6: Ablation of T5 (+Memory+Control) against individual components. CRR will be reported for each failure category upon completion of the evaluation. [Placeholder — to be filled]

Condition	**A**	**B**	**C**	**D**	**E**
+Memory	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*
+Control	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*
+Memory+Control (T5)	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*	*[TBD]*

C SFT Data Construction Details

For the SFT training data construction, corrected trajectories will be generated by prompting GPT-4o with the following template:

System: You are an expert agent that always produces correct trajectories. Given the task below, generate the correct sequence of actions and tool calls that leads to the gold answer.
User: Task: [task description]
Gold Answer: [gold answer]
Agent Trajectory (failed): [original trajectory]
Please generate the corrected trajectory with explanations.

Verification: SFT training is planned as the next phase. Trajectory quality verification protocol (including human spot-checks of corrected trajectories) is planned as part of the SFT data construction pipeline.

D Annotation Protocol Details

Formal failure annotation with inter-annotator agreement measurement is planned as part of the complete evaluation pipeline. The planned protocol follows the two-phase design in §**??**: Phase 1 (pilot) with 60 failure traces and Phase 2 (formal) with 140 additional cases, targeting Cohen's $\kappa \geq 0.70$.

E Additional Baseline Details

Human ceiling: Human expert evaluation on PinchBench and AgentBench-OC is planned but not yet conducted. This will provide an upper-bound reference for ceiling effect analysis.

Inter-condition correlation: Pearson correlation of paired deltas across conditions is planned as part of the full evaluation. This analysis will confirm that conditions targeting different failure categories capture distinct repair mechanisms.